# QuickDrop CodeJam 2026 - Guia de Soluciones

Explicacion completa paso a paso con ejemplos: Solucion 1 (Base) + Solucion 2 (Pedidos Premium)

## 1. EL PROBLEMA Y LA FORMULA DE COSTOS

En QuickDrop debemos repartir pedidos en una grilla de 20x20 cuadras con distancia Manhattan (|x1 - x2| + |y1 - y2|). Todos los vehiculos salen del deposito a las 08:00 y al terminar regresan al deposito.

```
Costo Total = (Cuadras_Totales x $3) + (Vehiculos_Usados x $60)
            + (Makespan_Minutos x $1) + (Pedidos_Fallados x $200)
            + [Evento 2: Suma(Minutos_Espera_Premium) x $1]
```

- **El dilema (Trade-Off): Si usamos 1 solo auto ahorramos el costo fijo ($60), pero tarda** casi 600 minutos en hacer todo (el Makespan nos cuesta $600). Si usamos 5 autos bajamos el tiempo, pero pagamos $300 en autos y multiplicamos por 5 los viajes de ida y vuelta al deposito.
- **Como lo solucionamos: El algoritmo prueba automaticamente desde k = 1 hasta k = N** vehiculos en 0.05 segundos, calcula el costo real con la formula oficial y elige la cantidad de autos que da el menor costo.

## 2. ARQUITECTURA DE ARCHIVOS SEPARADOS

- **1) geometria.py: Calcula el angulo polar (0 a 360 grados) de cada pedido respecto al** deposito, su distancia Manhattan y recorta los huecos vacios donde no hay pedidos.
- **2) ruteo.py (Solucion 1 - Sin Premium): Divide los pedidos en franjas por angulo util y** optimiza cada ruta con la metaheuristica GRASP + 2-Opt (Tesis Iriarte, UBA 2018) para eliminar cruces.
- **3) ruteo_premium.py (Solucion 2 - Con Premium): Resuelve el Evento 2 agregando inversion** de sentido de ruta, GRASP ponderado por urgencia, 2-Opt sensible a latencia, operador Or-Opt (Relocate) y vehiculo express.
- **4) main.py y main_premium.py: Leen la carpeta Datos/, comparan todos los k vehiculos y** guardan el JSON ganador en respuestas/plan.json.

## 3. FASE 1 (geometria.py): ANGULOS 0-360 Y RECORTE DE HUECOS

Como el deposito puede cambiar de lugar (de (0,0) a (10,10) o cualquier punto), calculamos las diferencias dx = x_pedido - x_deposito, dy = y_pedido - y_deposito y obtenemos el angulo de 0 a 360 grados:

```
angulo = math.degrees(math.atan2(y - dep_y, x - dep_x)) % 360.0
```

- **Desempate de angulos iguales: Si dos pedidos estan en el mismo angulo (misma calle o** diagonal desde el deposito), se ordena primero el mas cercano al deposito para visitarlos de paso sin retroceder.

**Ejemplo del Recorte de Huecos Vacios (Tu idea central):**

Que pasa si hay pedidos de 0 a 30 grados, NO hay nadie entre 30 y 61 grados (31 grados vacios), y vuelve a haber pedidos de 61 a 90 grados?

1. Los pedidos ya estan ordenados por angulo: [..., 25, 28, 30, 61, 64, 70, ...].
2. El algoritmo resta cada angulo con el anterior: entre 30 y 61 detecta un salto de 31 grados (> 12 grados de umbral).
3. 'Corta con tijera' esos 31 grados vacios y une las zonas ocupadas: quedan 30 + 29 = 59 grados utiles reales.
4. Si probamos con 2 autos (29.5 grados utiles por auto):

- Auto 1: atiende de 0 a 30 grados.
- (Salta el desierto vacio de 30 a 61 grados).
- Auto 2: atiende de 61 a 90 grados. Ningun auto desperdicia su franja en una zona sin clientes.

## 4. SOLUCION 1 (ruteo.py): GRASP + 2-OPT (PROBLEMA BASE)

Una vez dividida la ciudad en franjas sin que los autos se crucen entre si, cada auto ordena sus paradas con la metaheuristica de la tesis de Iriarte (UBA 2018):

- **Fase 1 - Construccion GRASP: Si un auto va siempre al vecino mas cercano de forma miope,** al final del recorrido le quedan pedidos aislados y cruza toda la ciudad. GRASP arma una lista (RCL) con los mejores candidatos cercanos y prueba 35 construcciones adaptativas.
- **Fase 2 - Busqueda Local 2-Opt: Toma la ruta del auto y prueba invertir cada tramo** [i..j]. Cada vez que dos lineas se cruzan en la grilla, 2-Opt invierte el tramo y las desenreda hasta llegar al minimo de cuadras.

**Resultado Real en el Mapa de 40 Pedidos (Carpeta Datos/):**

- Con 1 auto: 124 cuadras ($372) + 572 min ($572) + $60 auto = $1.004
- Con 2 autos: 154 cuadras ($462) + 346 min ($346) + $120 autos = $928 <-- OPTIMO ELEGIDO
- Con 3 autos: 192 cuadras ($576) + 293 min ($293) + $180 autos = $1.049
- Con 4 autos: 242 cuadras ($726) + 266 min ($266) + $240 autos = $1.232
- Con 5 autos: 274 cuadras ($822) + 266 min ($266) + $300 autos = $1.388

Conclusion: Usar 2 vehiculos ahorra 226 minutos de tiempo respecto a 1 vehiculo, logrando el costo minimo de $928.

## 5. SOLUCION 2 (ruteo_premium.py): ALGORITMO PARA PEDIDOS PREMIUM

En el Evento 2 aparece el campo 'premium' en los pedidos y 'costo_por_minuto_premium' ($1/min) en el mundo. Cada minuto que un cliente Premium espera desde las 08:00 hasta que el auto llega a su puerta nos suma $1 de penalizacion.

Esto convierte el problema en un Minimum Latency VRP (Ruteo con Minima Latencia). Implementamos 4 mejoras clave en ruteo_premium.py:

### Mejora 1: Inversion de Giro del Lazo (Ahorro con 0 cuadras extra)

**Ejemplo de Inversion de Giro:**

Un auto sale del deposito (0,0) y hace un recorrido en forma de gota: [P1 -> P2 -> ... -> P10] volviendo a (0,0).
- En sentido horario [P1 a P10] recorre 60 cuadras y llega a P10 en el minuto 195.
- En sentido antihorario [P10 a P1] recorre EXACTAMENTE las mismas 60 cuadras, pero llega a P10 en el minuto 18.

Si P10 es Premium, dar vuelta el sentido del recorrido ahorra 177 minutos ($177) sin gastar ni una cuadra extra.

### Mejora 2: GRASP Ponderado por Prioridad

**Ejemplo de Distancia Ponderada:**

Al salir a las 08:00, el auto tiene un pedido Normal a 4 cuadras y un pedido Premium a 7 cuadras.
Multiplicamos la distancia de los Premium por un factor de urgencia (0.0, 0.35 y 0.60). Con factor 0.35, la distancia aparente del Premium baja a 7 x 0.35 = 2.45 cuadras.
Asi el algoritmo atrae naturalmente al vehiculo hacia los clientes Premium en los primeros minutos de la manana.

### Mejora 3: 2-Opt Sensible a Latencia + Operador Or-Opt (Relocate)

**Ejemplo Numerico del Operador Or-Opt (Relocate):**

En vez de mirar solo cuadras, ruteo_premium.py evalua en cada movimiento: (Cuadras x $3) + (Minutos_Premium x $1).

El operador Or-Opt toma un pedido Premium que estaba en el puesto 6 de la ruta (minuto 95) y prueba adelantarlo al puesto 2 (minuto 30):

- Desviarse para atenderlo antes suma 4 cuadras (+4 x $3 = +$12 de costo en distancia).
- Pero el cliente Premium recibe su paquete 65 minutos antes (-65 x $1 = -$65 de ahorro en espera).
- Resultado neto: +$12 - $65 = -$53. El algoritmo acepta el cambio y ahorra $53.

Ademas, cuando NO hay pedidos premium, Or-Opt igual funciona puliendo paradas sueltas (bajo de 126 a 124 cuadras con 1 auto).

### Mejora 4: Competencia de 4 Modos de Flota (Vehiculo Express y Paralelo)

Para cada k, el Algoritmo 2 pone a competir 4 repartos: (1) Angulo util, (2) Balanceado, (3) Vehiculo Express dedicado exclusivamente a los Premium y (4) Corte equilibrado de Premiums para que todos los autos atiendan clientes Premium en paralelo a las 08:00.

**Prueba con Pedidos Premium activos (Pedidos 2 y 6 como Premium):**

- Costo del plan oficial de referencia: $1.124.
- Ruta generada por nuestro Algoritmo 2: [2 (Premium), 8, 3, 6 (Premium), 7, 9, 4, 1, 10, 5].

Sale a las 08:00 directo al Premium 2, sube por 8 y 3 directo al Premium 6, y a la vuelta barre los normales, bajando el costo a $824 ($300 de ahorro).

## 6. RESUMEN PARA EL PITCH FRENTE AL JURADO (5 MINUTOS)

- **1) Toma de decisiones: Elegimos Cluster-First, Route-Second combinando barrido polar** 0-360 con recorte de huecos vacios y la metaheuristica GRASP + 2-Opt (UBA) para evitar la miopia del vecino mas cercano.
- **2) Cantidad de vehiculos: Modelamos el equilibrio entre el costo fijo por auto ($60 +** retorno al deposito) y el ahorro en makespan ($1/min), evaluando k = 1..N automaticamente.
- **3) Adaptabilidad al Evento Premium: Gracias al diseno modular, extendimos** ruteo_premium.py con inversion de giro, GRASP ponderado y Or-Opt guiado por latencia, funcionando tanto con datasets normales como con datasets Premium.